

Demolab

An agent-operated lab notebook for computational science.

demolab.eoinmurray.info

Eoin Murray · lab group talk

What demolab is, and isn't

It is

- **Self-publishing:** repo $\rightarrow$ site + PDFs.
- **Reproducible:** numbers read from the run.
- **Agent-operated:** plain language.
- **Yours:** your science and git history.

It isn't

- Another hand-maintained notebook.
- A library you write code against.
- Tied to Python.
- Autonomous — you stay in the loop.

A structure and a workflow, not a framework.

The gap

- In my circles, **software engineers** use coding agents constantly.
- **Academics** — far less.
- Yet our work is exactly what they're good at: **code + write-up + literature**.
- Meanwhile results rot: scripts, stray plots, numbers retyped into the paper.
- **demolab** is an attempt to close that gap.

Coding agents — what are they?

- AI assistants that read a repo and **do the work** — not just autocomplete.
- Run in your terminal or editor: read files, run commands, edit, verify.
- You drive them in **plain language**; they follow instructions in the repo.
- Prominent ones: **Claude Code**, **Cursor**, **Copilot** (agent mode), **Gemini CLI**, **aider**.
- demolab is **operated by** one — so there's no web dev and no build config for you.

Before and after coding agents

Before

- You hand-write boilerplate, plots, build config.
- Docs and code quietly drift apart.
- Reproducing an old result is archaeology.

After

- You describe the result; it scaffolds and runs it.
- Prose, figures, and numbers stay in sync.
- Roughly **5× the throughput**, more projects in flight.

The plumbing stops being your job.

Problems with coding agents

- **Confidently wrong:** they fabricate APIs, numbers, and citations.
- **No memory:** they forget context between sessions unless it's written down.
- **Weak judgement:** they won't tell you the experiment itself is a bad idea.
- **Drift & noise:** code and prose fall out of sync; they over-produce.

demolab's answer: guardrails — numbers from the run, a fixed structure, git provenance on every result, and lint / doctor / red-team checks. Trust the structure.

The shape of a demolab repo

<code>tools/</code>	the science – models & solvers
<code>experiments/</code>	the runners – <code>expNNN.py</code> per experiment
<code>writings/</code>	the writeups – one <code>.typ</code> per entry
<code>temp/</code>	the raw run data – gitignored, big
<code>artifacts/</code>	the distilled record – <code>data/</code> + <code>pdfs/</code> (committed)
<code>demolab.yaml</code>	branding + collections (optional)
<code>demolab-engine/</code>	the engine (black box you never edit)

Guides and runbooks

Guides — the rules, always on

- **RULES**: the tool $\leftrightarrow$ experiment contract.
- **HOUSESTYLE**: how a writing reads.
- plus **STRUCTURE**, **GLOSSARY**, **SLIDES**, **SUPPORT**.

Runbooks — 14 named procedures

- **LINT · DOCTOR · RED-TEAM**: audits.
- **MIGRATE-CODE · UPDATE**: big moves.
- **GETTING-STARTED**: the first run.

Trigger any by name, in plain language; the runbook keeps the agent on rails.

Driven by a handful of commands

<code>task install</code>	set up (Python deps via uv)
<code>task run -- exp000</code>	run an experiment end-to-end
<code>task dev</code>	serve the site, live-reload on save
<code>task build</code>	website + a PDF per entry + a book
<code>task test</code>	run the test suite

Use any stack you want

- Ships set up for **Python**: `uv`, tools + runners as `.py`, `pytest`.
- But the contract is **files, not a language**: a tool just writes `numbers.json` + data.
- So switching is a conversation: “**migrate the stack to MATLAB**” — the agent follows a runbook.
- Works for **MATLAB · R · Julia · Octave**; the Typst publishing and the contract stay put.
- Keep Python only for plotting, or drop it entirely — your call.

Try it

demolab.eoinmurray.info

- Open a coding agent in a new, empty folder.
- Point it at the repo; approve as it sets things up.
- It hands you a live URL and your first experiment.

Hand it the loop, keep the science yours.